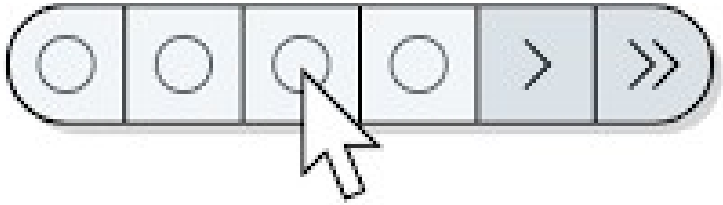
QuerySets e Paginação



Prof. Dr. João Paulo Lemos Escola
Copyright© 2026

O que veremos?

- QuerySets:
  - Permitem consultas avançadas de dados.
- Paginação:
  - Permitem limitar o número de registros exibidos por tela, incluindo uma lista de páginas.

1) QuerySets

- Query = consulta
- Set = conjunto
- Coleção de dados do BD
- Tratado como uma lista de objetos Python
- Permite realizar filtragens facilitadas nos dados

all

- Método que retorna todos os registros

```python
def members(request):
    mymembers = Member.objects.all().values()
    template = loader.get_template('all_members.html')
    context = {
        'mymembers': mymembers,
    }
    return HttpResponse(template.render(context, request))
```

```html
<ul class="list-group">
    {% for x in mymembers %}
    <li class="list-group-item">
        <a href="details/{{ x.id }}">{{ x.firstname }} {{ x.lastname }}</a>
        <a href="delete/{{ x.id }}" class="btn btn-danger"><i class='fa-solid fa-trash-can'></i></a
        <a href="update/{{ x.id }}" class="btn btn-info"><i class='fa-solid fa-pen'></i></a>
    </li>
    {% endfor %}
</ul>
```

Por coluna

- **values_list** define a coluna a retornar:

```
def members(request):
    mymembers = Member.objects.values_list('firstname', flat=True)
    template = loader.get_template('all_members.html')
    context = {
        'mymembers': mymembers,
    }
    return HttpResponse(template.render(context, request))
```

```
<ul class="list-group">
    {% for x in mymembers %}
    <li class="list-group-item">
        <a href="details/{{ x.id }}">{{ x }}</a>
```

Por linha

- **filter** define a linha a retornar:

```
def members(request):
    mymembers = Member.objects.filter(firstname='a').values()
    template = loader.get_template('all_members.html')
    context = {
        'mymembers': mymembers,
    }
    return HttpResponse(template.render(context, request))
```

```
<ul class="list-group">
    {% for x in mymembers %}
    <li class="list-group-item">
        <a href="details/{{ x.id }}">{{ x.firstname }} {{ x.lastname }}</a>
```

Filter

- Permite realizar consultas:

```
mydata = Member.objects.filter(firstname='Emil').values()
```

- Equivalente a:

```
SELECT * FROM members WHERE firstname = 'Emil';
```

and

- Filtrando utilizando dois campos:

```
mydata = Member.objects.filter(lastname='Refsnes', id=2).values()
```

- Equivalente a:

```
SELECT * FROM members WHERE lastname = 'Refsnes' AND id = 2;
```

or

- Filtrando com múltiplas consultas:

```
mydata = Member.objects.filter(firstname='Emil').values() | Member.objects.filter(firstname='Tobias').values()
```

- Utilizando expressão Q:

```
mydata = Member.objects.filter(Q(firstname='Emil') | Q(firstname='Tobias')).values()
```

- Equivalente a:

```
SELECT * FROM members WHERE firstname = 'Emil' OR firstname = 'Tobias';
```

Field Lookups

- Pesquisa por campos
- Sintaxe: nome do campo, seguido de dois *underlines*, seguido da palavra-chave:

```
.filter(firstname__startswith='L');
```

Is the same as the SQL statement:

```
WHERE firstname LIKE 'L%'
```

Exemplos

```
mydata = Member.objects.filter(firstname__contains='bias').values()
```

```
mydata = Member.objects.filter(lastname__icontains='ref').values()
```

```
mydata = Member.objects.filter(firstname__endswith='s').values()
```

```
mydata = Member.objects.filter(id__gt=3).values()
```

```
mydata = Member.objects.filter(id__range=(2, 4)).values()
```

Field Lookups Reference

A list of all field look up keywords:

Keyword	Description
contains	Contains the phrase
icontains	Same as contains, but case-insensitive
date	Matches a date
day	Matches a date (day of month, 1-31) (for dates)
endswith	Ends with
iendswith	Same as endswidth, but case-insensitive
exact	An exact match
iexact	Same as exact, but case-insensitive
in	Matches one of the values
isnull	Matches NULL values
gt	Greater than
gte	Greater than, or equal to

hour	Matches an hour (for datetimes)
lt	Less than
lte	Less than, or equal to
minute	Matches a minute (for datetimes)
month	Matches a month (for dates)
quarter	Matches a quarter of the year (1-4) (for dates)
range	Match between
regex	Matches a regular expression
iregex	Same as regex, but case-insensitive
second	Matches a second (for datetimes)
startswith	Starts with
istartswith	Same as startswith, but case-insensitive
time	Matches a time (for datetimes)
week	Matches a week number (1-53) (for dates)
week_day	Matches a day of week (1-7) 1 is sunday
iso_week_day	Matches a ISO 8601 day of week (1-7) 1 is monday
year	Matches a year (for dates)
iso_year	Matches an ISO 8601 year (for dates)

Order By

- Define a ordem da listagem:

```
mydata = Member.objects.all().order_by('firstname').values()
```

```
mydata = Member.objects.all().order_by('-firstname').values()
```

- Equivalente a:

```
SELECT * FROM members ORDER BY firstname;
```

```
SELECT * FROM members ORDER BY firstname DESC;
```

Order By Múltiplo

- Basta adicionar mais um valor na função:

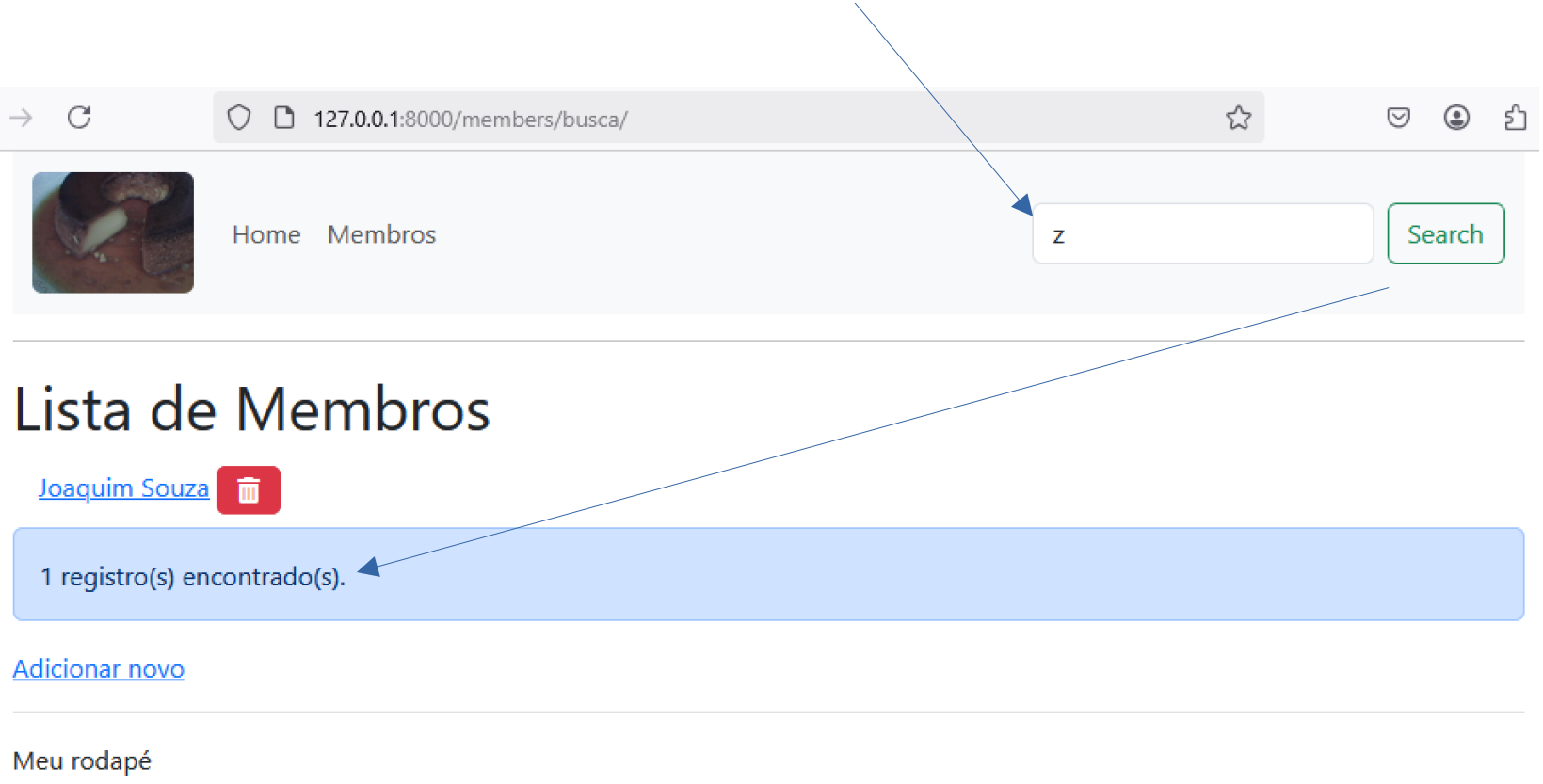
```
mydata = Member.objects.all().order_by('lastname', '-id').values()
```

```
SELECT * FROM members ORDER BY lastname ASC, id DESC;
```

Barra de pesquisa

- Vamos fazer nossa barra de pesquisa funcionar;
- Para isso, vamos criar uma rota (urls.py) para receber as pesquisas dos usuários;
- A listagem de membros será exibida de acordo com a busca realizada.

Resultado esperado:



master.html

```
<form class="d-flex" role="search" method="post" action="/members/busca/">
    {% csrf_token %}
    <input class="form-control me-2" type="search" placeholder="Search" aria-label="Search" name="busca" value="{{ texto_busca }}">
    <button class="btn btn-outline-success" type="submit">Search</button>
</form>
```

Texto da última busca realizada.

urls.py

```
5   urlpatterns = [
6       path('', views.main, name='main'),
7       path('members/', views.members, name='members'),
8       path('members/busca/', views.busca, name='busca'),
9       path('members/details/<int:id>', views.details, name='details'),
10      path('members/add/', views.add, name='add'),
11      path('members/add/addrecord/', views.addrecord, name='addrecord'),
12      path('members/delete/<int:id>', views.delete, name='delete'),
13  ]
```

views.py

Campo de busca

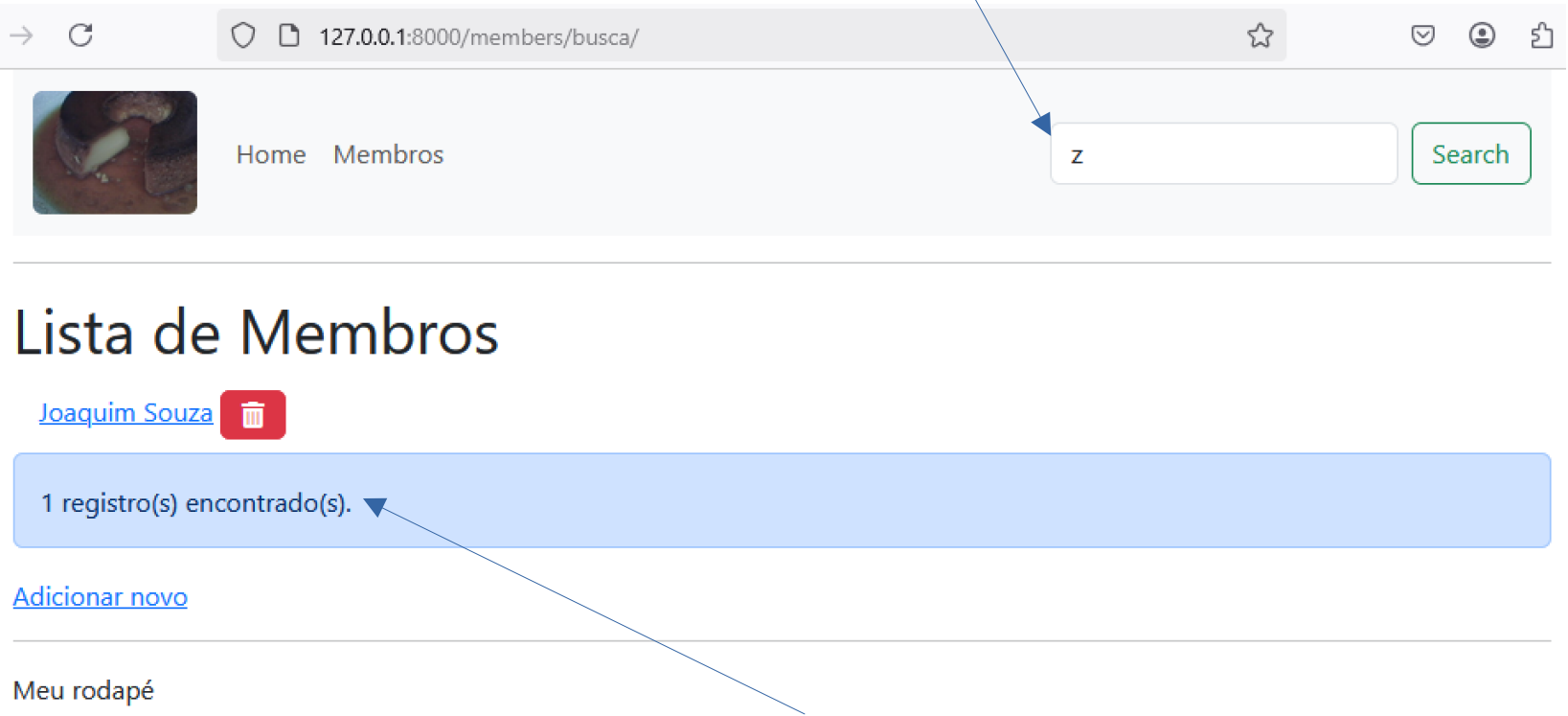
```
5    from django.db.models import Q
```

```
17   def busca(request):
18       texto_busca = request.POST['busca']
19       dados = Member.objects.filter(
20           Q(firstname__icontains=texto_busca) | Q(lastname__icontains=texto_busca)
21       ).values()
22       t = loader.get_template('index.html')
23       context = {
24           'mymembers': dados,
25           'texto_busca': texto_busca,
26           'qtd': len(dados),
27       }
28       return HttpResponse(t.render(context, request))
```

index.html

```
{% extends "master.html" %}

{% block title %}
Listagem de membros
{% endblock %}

{% block content %}

<h1>Lista de Membros</h1>
<ul class="list-group list-group-flush">
    {% for x in mymembers %}
    <li class="list-group-item">
        <a href="details/{{x.id}}">{{x.firstname}} {{x.lastname}}</a>
        <a href="delete/{{x.id}}" class="btn btn-danger fa-solid fa-trash-can"></a>
    </li>
    {% endfor %}
</ul>

{% if qtd %}
<div class="alert alert-primary" role="alert">
    {{ qtd }} registro(s) encontrado(s).
</div>
{% endif %}

<a href="add">Adicionar novo</a>

{% endblock %}
```

Campo
name="busca"



Variável qtd

2) Paginação

- O Django provê funcionalidades para facilitar o processo de paginação
- Vamos configurar nossa view:

```
6   from django.core.paginator import Paginator
```

```
9   def members(request):
10      mymembers = Member.objects.all().values()
11      template = loader.get_template('all_members.html')
12
13      paginator = Paginator(mymembers, 2)
14      page_number = request.GET.get("page")
15      page_obj = paginator.get_page(page_number)
16
17      context = {
18          # 'mymembers': mymembers,
19          "page_obj": page_obj,
20      }
21      return HttpResponse(template.render(context, request))
```

Paginação no bootstrap

Previous 1 2 3 Next

```
<nav aria-label="Page navigation example">
  <ul class="pagination">
    <li class="page-item">
<a class="page-link" href="#">Previous</a></li>
    <li class="page-item">
<a class="page-link" href="#">1</a></li>
    <li class="page-item">
<a class="page-link" href="#">2</a></li>
    <li class="page-item">
<a class="page-link" href="#">3</a></li>
    <li class="page-item">
<a class="page-link" href="#">Next</a></li>
  </ul>
</nav>
```

Substituir o vetor de membros

```
{% for x in page_obj %}
<li class="list-group-item">
    <a href="details/{{x.id}}">{{x.firstname}} {{x.lastname}}</a>
    <a href="delete/{{x.id}}" class="btn btn-danger fa-solid fa-trash-can"></a>
</li>
{% endfor %}
```

mymembers contem todos os registros;

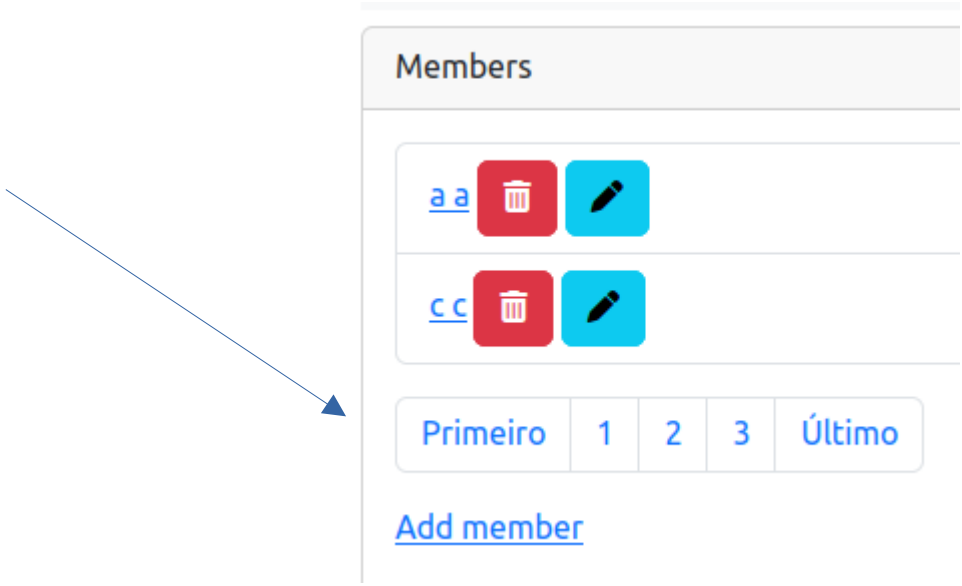
page_obj contém somente os registros da página atual;

all_members.html

- Adicione a barra de paginação:

```
27      <nav>
28          <ul class="pagination mt-3">
29              <li class="page-item"><a class="page-link" href="?page=1">Primeiro</a></li>
30              {% for p in page_obj.paginator.page_range %}
31              <li class="page-item"><a class="page-link" href="?page={{p}}">{{p}}</a></li>
32              {% endfor %}
33              <li class="page-item"><a class="page-link" href="?page={{ page_obj.paginator.num_pages }}">Último</a></li>
34          </ul>
35      </nav>
36
37      <p class='mt-3'>
38          <a href="add/">Add member</a>
39      </p>
```

Veja o resultado



O que vimos?

- QuerySets
- Paginação

Para praticar...

- Inclua o recurso de pesquisa para buscar itens na listagem de equipamentos:
  - Quando o usuário está na listagem de membros, deve continuar como está;
  - Quando o usuário está na listagem de equipamentos, deve pesquisar somente equipamentos.

Referências

- w3schools.com/django/django_queryset.php
- https://docs.djangoproject.com/en/4.2/topics/pagination/
- https://getbootstrap.com/docs/5.1/components/pagination/